

Cover Letter for TOSEM Submission

TyFlow: A Type-Aware Approach to Neural Code Models

Manuscript TOSEM-2026-0076 • Major Revision, decision of June 16, 2026

We thank the reviewers and the editor for their valuable comments. We have revised our paper accordingly. Every reviewer comment is quoted below in italics, followed by our response and by a summary of the changes we made in the paper; section and page numbers refer to the revised manuscript (45 pages).

Summary of changes. The revision adds two Java benchmarks (HumanEval-Java and TransCoder-GFG) that enlarge the Java evaluation to 186 tasks, a statistical analysis with 95% intervals and exact paired tests (App. D), and a task-level failure taxonomy (App. C). It also adds runtime and pruning measurements of the constrained decoder (RQ2). The comparison with other generation methods now covers three decoder-only models of 7B–30B parameters (Table 3) and four reliability-oriented approaches (RQ3, Table 5). A new Limitations subsection (Sec. 6.3) discusses the encoder-decoder dependence, dynamically typed languages, and type-system expressiveness.

Response to the Editor

Editor’s comment.

All the reviewers agree that the paper investigates an important topic. However, reviewers also point out major concerns that need to be addressed and responded to, hence my recommendation for a Major Revision. In the following, I have provided some of the critical concerns raised by the reviewers.

- *Missing failure analysis*
- *Limited evaluation benchmarks*
- *Missing discussion on scalability*
- *Missing comparison with larger and more recent LLMs*
- *Missing comparison with state-of-the-art techniques*
- *Missing analysis of the system’s cost*

Details on the above points are in the reviews of the reviewers involved in the process. While preparing a Major Revision, please make sure to respond to all the other reviewers’ comments and questions, not just the points summarized above.

Response.

Thanks. Each of the six concerns is addressed with new experiments or new analyses, as summarized below:

- **Failure analysis.** Every test task of the three Java benchmarks is classified into four categories in App. C (p. 42–43, Table 8), with two representative cases; the dominant residual mode of TyFlow is well-typed but semantically wrong code. Detailed response: [Comment 1.6](#).
- **Benchmarks.** HumanEval-Java (146/16) and TransCoder-GFG (414/103) are added, so the Java evaluation spans 186 test tasks from three independent sources, analyzed jointly in App. D (p. 44). Detailed responses: [Comment 1.3](#) and [Comment 2.1](#).
- **Scalability.** RQ2 measures the pruning strength of each component, the beam-exhaustion behavior, and the per-step and per-task runtime (up to $2.74\times$ the unconstrained base); scaling to richer type systems is discussed in the Limitations subsection. Detailed responses: [Comment 1.2](#) and [Comment 2.2](#).
- **Larger and more recent LLMs.** Zero- and few-shot pass@1 results for MiMo-7B, Qwen3-14B, and Qwen3-30B-A3B on all four benchmarks are added (Table 3, p. 26): on Java they outperform the fine-tuned 2B models, on SuFu none solves a single task zero-shot. Detailed responses: [Comment 1.5](#) and [Comment 2.3](#).
- **State-of-the-art techniques.** RQ3 is rewritten as a five-method comparison on MBJP (rejection sampling, SynCode, Repilot, iterative compiler repair) on one leak-clean frozen checkpoint; none of the four external methods significantly improves functional correctness, while TyFlow-2B is the best of all compared methods. Detailed responses: [Comment 3.1](#) and [Comment 3.2](#).
- **System cost.** The runtime analysis is reported in RQ2 together with the component ablation it belongs to: per-step and per-task wall time of the same 220M model in the four configurations, with an explanation of where each cost comes from and why syntactic pruning even *lowers* the per-task total. For the compared external methods, the RQ3 setup states each method’s computational budget. Detailed responses: [Comment 2.2](#) and [Comment 3.3](#).

Beyond these six cross-cutting concerns, we respond to every other reviewer comment and question individually in Comments 1.1–3.6 below.

Response to Reviewer 1

We thank the reviewer for the positive assessment of the formalization, its generality over grammar-based approaches, and the ablation study.

Comment 1.1.

The example language ($\lambda \rightarrow$) is extremely simple. While it serves the pedagogical purpose well, readers may reasonably wonder whether the approach scales to languages with subtyping, polymorphism, type inference, overloading, or mutable state. The paper partially addresses this by evaluating on Java, but the overview could better acknowledge the gap between $\lambda \rightarrow$ and real languages.

The restriction to first-order unification deserves further discussion. Higher-order unification arises naturally in languages with higher-order functions and polymorphism. The authors should discuss what happens when this restriction is violated and what classes of type systems are excluded.

[Weaknesses:] *The restriction to first-order unification and encoder-decoder architectures limits near-term applicability.*

Response.

Thanks. We answer the three questions separately.

(1) The running example and richer type systems.

The difference from real languages. The example language is deliberately simple and is used only to demonstrate how TyFlow converts typing rules into synthesis rules. It contains Boolean and function types with explicitly annotated, monomorphic bindings. Unlike mainstream languages, it does not include features like subtyping, polymorphism, type inference, overloading, or mutable state. We now acknowledge this gap explicitly in the overview.

Extension to richer type systems. TyFlow is not tied to this example language. To support a richer statically typed language, we define its program, type, and context terms, express its typing rules as constrained Horn clauses, and implement the required computable constraints. The transformation from typing rules to synthesis rules remains unchanged. Because this language-specific implementation requires substantial engineering effort, we currently instantiate TyFlow only for Java and SuFu.

(2) The first-order unification restriction.

Why we use it. To apply a typing rule, TyFlow needs unification to terminate, decide whether the rule conclusion matches the current goal, and return a principal substitution for the remaining premises. First-order unification provides these guarantees: it is decidable and returns a most-general unifier that is unique up to variable renaming. We therefore use first-order unification as a sufficient condition.

Higher-order functions do not require higher-order unification. A higher-order function takes or returns a function value, and the type checker needs to determine the type of that value. Higher-order unification is needed only when the unknown metavariable to be solved in the typing rule is itself a function, type constructor, or predicate. The following two terms show the difference:

$$\text{Arrow}(\alpha, \beta) \quad \text{versus} \quad F(\text{Int}) = \text{List}(\text{Int}).$$

In the first term, *Arrow* is fixed. Only α and β are unknown, so this is still a first-order problem. In the second term, *F* is such an unknown, so it is a higher-order unification problem.

ML is an example. It supports higher-order functions and parametric polymorphism, but Hindley–Milner type inference uses first-order unification. Java provides the same distinction. A simplified typing rule for a higher-order API such as `Stream.map` is:

$$\frac{\Gamma \vdash s : \text{Stream}\langle A \rangle \quad \Gamma \vdash f : \text{Function}\langle A, B \rangle}{\Gamma \vdash s.\text{map}(f) : \text{Stream}\langle B \rangle}.$$

The program passes a function as a value, but `Stream` and `Function` are fixed type constructors. Rule application only needs to instantiate the ordinary type terms *A* and *B*. Thus, this higher-order language feature still requires only first-order unification.

Support for real-world type systems. Java supports function values, but its type checking does not require unification over unknown type-level functions, type constructors, or predicates. Its program, type, and context syntax can be represented as first-order terms. The Java Language Specification handles generic inference through constraint reduction, incorporation, and resolution over inference variables.¹ Features such as subtyping, generic bounds, overload applicability, and lambda target typing can therefore be handled through typing rules and language-specific constraints. Supporting these features requires substantial implementation work, but it does not require higher-order unification as long as rule application only instantiates ordinary term variables.

Dependent-type systems such as Lean are different because elaboration may need to infer an unknown type constructor or predicate. Lean therefore uses higher-order unification together with heuristics and backtracking.² The current TyFlow solver cannot directly support these cases. A decidable fragment with principal unifiers, such as higher-order patterns, could be incorporated, but this would require a new unifier. General higher-order unification remains outside our current framework.

(3) The architecture restriction. The current implementation cannot be moved directly because the encoder reads the changing synthesis goal. A decoder-only version would have to encode this goal at each step, and its cached states could not be reused. We discuss this issue and the decoder-only evaluation in [Comment 2.3](#).

¹*The Java Language Specification*, Java SE 25, Sec. 18, “Type Inference.”

²Leonardo de Moura, Jeremy Avigad, Soonho Kong, and Cody Roux. “Elaboration in Dependent Type Theory.” CADE-25, 2015.

Changes made.

- **Added** Sec. 2 (p. 9): the overview now enumerates the gap between $\lambda \rightarrow$ and real languages (subtyping, polymorphism, type inference, overloading resolution, mutable state) and points to the evaluation and the Limitations discussion.
- **Clarified** Sec. 3 (p. 12): why TyFlow uses first-order unification, why higher-order functions do not by themselves require higher-order unification, and when the current solver no longer applies.
- **Added** Sec. 6.3, “Dependence on the encoder-decoder architecture” (p. 29): why the dynamic typing context is not directly portable to a decoder-only model.
- **Clarified** Sec. 6.3, “Type-system expressiveness” (p. 30): which features can use first-order terms and which require a different solver.

Revised manuscript, Sec. 2 (p. 9) – the acknowledged gap between the example language and real languages

The $\lambda \rightarrow$ language above serves for exposition: it is deliberately minimal, containing only the base type `bool`, function types, and explicitly annotated, monomorphic bindings. Real languages additionally involve subtyping, polymorphism, type inference, overloading resolution, and mutable state, none of which $\lambda \rightarrow$ exercises. The evaluation (Sec. 6) therefore runs TyFlow on Java and SuFu instances with fuller, though still restricted, type systems; the scope of the underlying rule framework is stated in Sec. 3, and how the approach would extend to richer type systems is discussed in Sec. 6.3.

Revised manuscript, Sec. 3 (p. 12) – the role and boundary of first-order unification

First-order unification always terminates. It either rejects two terms or returns a most-general unifier that is unique up to variable renaming. These guarantees let TyFlow apply a selected typing rule and update its remaining premises.

Supporting higher-order functions does not by itself require higher-order unification. The latter is needed only when an unknown to be solved is itself a function, type constructor, or predicate. If a typing rule creates such a problem, TyFlow’s current solver cannot handle it. A decidable fragment could be supported with a different unifier, but general higher-order unification remains outside the current framework.

Revised manuscript, Sec. 6.3, “Dependence on the encoder-decoder architecture” (p. 29) – the decoder-only adaptation path

The decision-sequence representation and synthesis rules can be used unchanged with a decoder-only model. [...] This requires recomputing the context’s key-value entries at each step and adapting cache management to the changing context. Implementing this decoding procedure and evaluating its runtime are directions for future work.

Revised manuscript, Sec. 6.3, “Type-system expressiveness” (p. 30) – type-system expressiveness as a limitation

For features that can be represented with first-order terms, such as many forms of subtyping, overloading, effects, and type classes, we need to implement the typing rules and solvers and then measure their cost. The current solver cannot handle a typing rule that requires higher-order unification, as discussed in Sec. 3.

Comment 1.2.

The practical implications of completeness deserve more discussion. Completeness says that for any well-typed program, there exists a synthesis derivation tree. But during generation, the LM must find the right sequence of decisions. If the LM makes a wrong choice early, it may be unable to recover. The paper does not discuss the branching factor of the search space or the likelihood of the LM making correct decisions in practice. Beam search partially addresses this, but the theoretical analysis gives no bounds on how many beams are needed.

The paper does not quantify the pruning rate in Section 5.3. What fraction of candidate tokens are eliminated at each level? This would help readers understand the practical search space reduction.

Response.

Thanks for this criticism. We now measure the constrained decoder, and we state explicitly what the completeness theorem does and does not imply for search.

(1) Completeness, branching factor, and beam behavior.

What completeness guarantees. The theorem is existential: it guarantees a derivation for every well-typed program, but it does not guarantee that the LM ranks a correct prefix within the beam, and it gives no bound on the required beam size. If all correct prefixes fall outside the beam, later decoding cannot recover them. Training on correct decision sequences is intended to assign higher probability to suitable choices.

Observed beam-10 behavior. We measure the effective branching factor as the number of active hypotheses present at each decoding step after pruning. At each step, the decoder applies grammar pruning, ranks the remaining expansions, and incrementally type-checks at most the 20 highest-scoring expansions to construct the next active beam. Theoretically, a finite beam can discard all correct prefixes even when the beam size is relatively large, so a subsequent active beam can become empty. However, this did not occur in the evaluated run where the beam width sets its maximum to 10: the active beam contained 7.97 hypotheses on average and never became empty. The average is lower than 10 because a completed program is moved to the output set, while rejected candidates do not fill an active slot.

(2) Pruning rates. Instrumenting the decoder on the SuFu tasks (beam 10, 330-step budget) gives: syntactic pruning removes 59.1% of all vocabulary proposals, and type pruning a further 8.2% (type checking is applied only to the high-probability candidates about to enter the beam, which is why its rate is lower). A final, complete type check on finished derivations rejects 21.2% of the completed candidates (113/533).

Changes made.

- **Added** Sec. 6.2.2 (p. 27): “Whether the constraints over-prune”: what completeness does and does not guarantee, the theoretical possibility of beam exhaustion, and the observed beam-10 behavior.
- **Added** Sec. 6.2.2 (p. 27): “How much the constraints prune”: instrumented rates (59.1% syntactic, 8.2% type, 21.2% final check) and the 330-step budget.

Revised manuscript, Sec. 6.2.2, “Whether the constraints over-prune” (p. 27) – completeness and the observed beam-10 behavior

Completeness guarantees that a well-typed program has a derivation, but it does not guarantee that the model ranks a correct prefix within a fixed beam or provide a bound on the required beam size. A finite beam can discard all correct prefixes, and the retained hypotheses may later have no valid continuation, leaving the active beam empty. [...] With beam 10, this number averages 7.97. [...] In our experiments, however, the active beam never became empty.

Revised manuscript, Sec. 6.2.2, “How much the constraints prune” (p. 27) – the measured pruning rates

To measure the pruning strength, we instrument the constrained decoder on the SuFu tasks (beam 10, 330-step budget): syntactic pruning removes 59.1% of all vocabulary proposals, and type pruning a further 8.2%. The lower rate is expected: type checking is more expensive, so it is applied only to the high-probability candidates about to enter the beam. Such candidates are rarely ill-typed, yet pruning them matters, for they would otherwise expand within the beam and crowd out well-typed derivations. Type checking during search is provisional (only completed AST fields are checked); the whole program is checked once when the derivation finishes, and this final check rejects 21.2% of the completed candidates.

Comment 1.3.

The SuFu test set has only ~58 tasks (20% of 290), and the Java test set has only ~61 tasks (10% of 608). These are very small test sets, and the results may not be statistically robust. The authors should report confidence intervals or conduct significance tests. A difference of 1–2 tasks passing or failing could shift pass@1 by several percentage points.

[Weaknesses:] *Small test sets raise concerns about statistical reliability. [...] Missing failure analysis and confidence intervals.*

Response.

Thanks for this comment. We address the dataset size and the statistical uncertainty separately.

(1) Expanded evaluation.

Benchmark coverage. We add two Java benchmarks from sources independent of MBJP. HumanEval-Java contains short algorithmic methods adapted from HumanEval, with a fixed split of 146 training and 16 test tasks. TransCoder-GFG contains numeric, string, and array problems drawn from the GeeksForGeeks programs released with TransCoder, with a fixed split of 414 training and 103 test tasks. Together with the original 67 MBJP test tasks, these

additions increase the Java evaluation from 67 to 186 test tasks across three benchmarks. SuFu remains a 58-task test set because the source repository contains 290 programs.

Experimental protocol. We evaluate the two added benchmarks at the 2B scale, comparing the baseline T5Gemma2-2B with our TyFlow-2B. Both systems use the same Java training tasks and are evaluated on the same fixed test tasks with beam 10 and ten candidates per task. We report pass@1, pass@10, FSP, and CER, so the evaluation covers functional correctness, candidate ranking, and compilation errors.

Results. The two added benchmarks show improvements on all four metrics:

- **HumanEval-Java.** pass@1 improves from 31.25% to 50.00%, pass@10 from 37.50% to 56.25%, and FSP from 6.50 to 4.75; CER falls from 24.38% to 1.30%.
- **TransCoder-GFG.** pass@1 improves from 19.42% to 30.10%, pass@10 from 33.01% to 46.60%, and FSP from 7.10 to 5.81; CER falls from 13.69% to 2.75%.

These results show that the improvements are not confined to the original MBJP split: they recur on two independently sourced Java benchmarks. The large CER reductions are consistent with the purpose of type-constrained generation, while the pass-rate and FSP gains show that the benefit is not limited to rejecting uncompileable outputs.

(2) Confidence intervals and significance tests. New App. D (p. 44) pairs every baseline–TyFlow run on the same tasks and reports a 95% interval and an exact two-sided paired test for every metric. For pass@1 and pass@10, we use Wilson intervals and McNemar tests; for FSP, a t -interval over per-task ranks and an exact sign test; and for CER, a Wilson interval over candidates and an exact sign test on per-task error fractions. The results show where the evidence is strong and where the small test sets still leave uncertainty:

- **Java.** At 220M, the Java evaluation contains only the 67 MBJP test tasks, and the small test set provides few paired observations: pass@10 improves from 20.90% to 28.36%, with 95% intervals of [12.88,32.07] for the baseline and [18.97,40.09] for TyFlow. The paired difference is not significant ($p = 0.359$), and only the CER difference is significant. At 2B, the two added benchmarks increase the Java test set to 186 paired tasks. On this larger test set, pass@10 improves from 33.33% to 46.24%, with 95% intervals of [26.96,40.38] and [39.22,53.41], respectively. The paired difference is significant ($p = 5.36 \times 10^{-4}$), as are the other three metric differences.
- **SuFu.** The small test set of 58 tasks also limits our ability to detect statistically significant improvements on this benchmark. At 2B, pass@10 improves from 37.93% to 48.28%, with 95% intervals of [26.56,50.80] for the baseline and [35.93,60.84] for TyFlow, but the paired difference is not significant ($p = 0.307$). At 220M, the pass@10 gain is larger: 32.76% to 53.45%. Its 95% intervals are [22.08,45.58] and [40.80,65.67], respectively, and the paired difference is significant ($p = 2.35 \times 10^{-3}$). All four paired differences are significant at 220M despite the same small test set.

Changes made.

- **Added** Sec. 6.1.1 (p. 24) and App. B (pp. 39–40): the sources, task characteristics, and fixed splits of HumanEval-Java and TransCoder-GFG.
- **Added** Sec. 6.2.1 (p. 25): the results of TyFlow-2B and its T5Gemma2-2B baseline on HumanEval-Java and TransCoder-GFG.
- **Added** App. D (pp. 44–45): 95% intervals and exact paired tests at the 220M and 2B scales (Tables 9–10), including the pooled 186-task Java analysis.

Revised manuscript, Sec. 6.1.1 (p. 24) – the two added Java benchmarks and their fixed splits

HumanEval-Java and TransCoder-GFG are two additional Java benchmarks that broaden the evaluation beyond MBJP. HumanEval-Java adapts the HumanEval problems to Java [...], and TransCoder-GFG collects the Java programs released with TransCoder [...]; both follow the task format of MBJP. HumanEval-Java uses a fixed split of 146 training and 16 test tasks, and TransCoder-GFG uses a fixed split of 414 training and 103 test tasks.

Revised manuscript, Sec. 6.2.1 (p. 25) – the results on the expanded benchmark set

TyFlow improves code generation accuracy at both scales and on every benchmark: [...] pass@1 rises [...] 31.25% to 50.00% (HumanEval-Java), and 19.42% to 30.10% (TransCoder-GFG) at 2B; pass@10 improves by 7.46–20.69 points on the same six benchmarks. [...] TyFlow also drops CER to 0.00% on SuFu and 0.45% / 1.30% / 2.75% on the three Java benchmarks; the small residual Java CER comes from static-analysis errors the type system does not capture.

Revised manuscript, App. D (p. 44) – the paired-test definitions and the 220M outcomes

We pair every CodeT5-220M baseline run with the matching TyFlow-220M run on the same tasks, and compute exact two-sided McNemar tests on pass@1 / pass@10 and exact two-sided sign tests on FSP and on per-task compilation-error fractions (CER). pass@1 and pass@10 report Wilson 95% intervals over tasks, CER over candidates, and FSP a t -interval over per-task ranks [...] with ten candidates per task. In the SuFu block all four paired differences favor TyFlow at the 0.05 significance level. In the Java block only CER is significant at $n = 67$, for two reasons: the pass-rate improvements themselves are modest at this scale (one additional task at pass@1, five at pass@10), and a 67-task test set yields few discordant pairs (11, 19, and 24 for pass@1, pass@10, and FSP), so the exact tests are underpowered; pooling the three Java benchmarks at the 2B scale restores significance for all four metrics.

Revised manuscript, App. D (p. 44) – the pooled 186-task Java analysis

The three Java benchmarks (MBJP, HumanEval-Java, TransCoder-GFG) all retain per-task records, so we pool them into one combined 186-task analysis rather than reporting each benchmark separately; the same tests and intervals as at the 220M scale apply. In the combined analysis all four paired differences favor TyFlow at the 0.05 significance level (Tab. 10).

Revised manuscript, App. D (p. 44) – the 2B SuFu outcomes

On SuFu only the compilation-error difference is significant at the 2B scale: TyFlow-2B has zero compilation errors on all 313 rendered candidates, so every one of the 58 tasks favors TyFlow in the exact sign test. The pass@1, pass@10, and FSP differences favor TyFlow in the same direction as at the 220M scale but are not significant at $n = 58$: the gains are six solved tasks at both pass@1

and pass@10, which leaves few discordant pairs (8 vs. 14 for pass@1, 9 vs. 15 for pass@10) and 30 nonzero FSP pairs (18 better, 12 worse), so the exact tests are underpowered, mirroring the 220M Java block.

Comment 1.4.

On Java, TyFlow-220M improves pass@1 from 10.45% to only 11.94%, less than 1.5 percentage points absolute. While pass@10 improves more substantially, the pass@1 gain is marginal, given the test set size. The paper should discuss why Java gains are smaller.

[Weaknesses:] *The Java results are modest, and the subset restriction introduces potential selection bias.*

Response.

Thanks for this comment. The Java results show gains in both functional correctness and compilability. We explain below why the improvement is smaller at pass@1 and how the other results help assess its value.

(1) Why the Java gains differ from SuFu. TyFlow improves code generation by preventing type errors during generation. Java is a mainstream programming language for which the pretrained baseline already has a strong foundation, whereas SuFu is a low-resource language. Consistent with this difference, the 220M Java baseline produces fewer compilation errors than the SuFu baseline (CER 35.52% versus 83.10%). This leaves fewer errors for our type constraints to address and therefore less room for improvement on Java.

(2) The value of the Java improvements. For a mainstream language such as Java, the improvement is substantial given that TyFlow and the baseline use the same training data. At 220M, TyFlow improves pass@10 by 7.46 percentage points (35.7% relative), a larger absolute gain than the 2.80-point improvement reported by GrammarT5-base over CodeT5-base on Java CONCODE, also at 220M.³ Our expanded experiments on additional Java benchmarks also show improvements over the corresponding baseline on all four metrics across all three Java benchmarks (Comment 1.3). Together, these results show consistent improvements over the baseline across the evaluated Java benchmarks and model scales.

(3) Subset Restriction. Our experiments use a supported Java subset, but tasks are selected by grammar rather than model performance. Both methods are evaluated on the same tasks, and TyFlow improves over the baseline across three independently sourced Java benchmarks and a SuFu benchmark. These consistent gains provide evidence that the improvement is not specific to one selected benchmark.

³Qihao Zhu et al. “GrammarT5: Grammar-Integrated Pretrained Encoder-Decoder Neural Model for Code.” ICSE 2024, Table 4. These gains are measured on different tasks and metrics.

Changes made.

- **Clarified** Sec. 6.2.1 (p. 25): why the 220M Java baseline’s lower compilation-error rate leaves less room for improvement than on SuFu.
- **Added** Sec. 6.2.1 (p. 25): the 2B results on MBJP and the two additional Java benchmarks, as described in [Comment 1.3](#).
- **Clarified** App. B (p. 37): the Java grammar coverage and the task selection used in the evaluation.

Revised manuscript, Sec. 6.2.1 (p. 25) – why Java leaves less room for improvement than SuFu

At 220M, the Java baseline has a lower CER than the SuFu baseline (35.52% versus 83.10%), leaving fewer compilation errors for type constraints to address. [...]

Revised manuscript, Sec. 6.2.1 (p. 25) – the Java results at 2B

TyFlow improves code generation accuracy at both scales and on every benchmark: pass@1 rises [...] 13.43% to 25.37% (MBJP), 31.25% to 50.00% (HumanEval-Java), and 19.42% to 30.10% (TransCoder-GFG) at 2B; pass@10 improves by 7.46–20.69 points on the same six benchmarks.

Revised manuscript, App. B (p. 37) – the implemented Java grammar and evaluated tasks

The implemented Java subset supports classes, methods, variables, arrays, generic types, operators, and control flow statements; it omits multithreading, lambda expressions, and complex reflection mechanisms. [...] Evaluation includes tasks whose constructs this grammar supports, so the reported results apply to this supported subset.

Comment 1.5.

The paper compares only with fine-tuned CodeT5 and T5Gemma2 variants. The absence of any comparison with modern LLMs is a notable gap. The authors could at least report zero-shot pass rates for state-of-the-art proprietary and open-weight LLMs on the same benchmarks to contextualize the absolute numbers.

[Weaknesses:] *No comparison with modern decoder-only large-scale code models.*

Response.

Thanks for this comment. We added the requested zero-shot comparison, plus a few-shot condition, for three larger open-weight decoder-only models (MiMo-7B, Qwen3-14B, Qwen3-30B-A3B) on all four benchmark families (Table 3, p. 27; protocol in the table note). The comparison contextualizes the absolute numbers:

- **SuFu (58 tasks).** None of the three models solves a single task zero-shot, and few-shot prompting reaches at most 8/58, whereas TyFlow-2B solves 21/58. The low-resource language is precisely where the type-guided representation delivers its advantage.
- **Java.** At pass@1, the three larger models solve 34–42 of 67 MBJP tasks, 3–14 of 16 HumanEval-Java tasks, and 38–52 of 103 TransCoder-GFG tasks. TyFlow-2B solves 17/67,

8/16, and 31/103, respectively. These results provide context for performance on Java under the respective generation settings.

- **A caveat we flag ourselves.** Few-shot prompting brings little or no gain over zero-shot on Java for these models, which suggests they may have seen this language extensively and may have been trained on these benchmarks directly; the revision therefore flags a possible data-leakage risk for the Java comparison.

Relative to its fine-tuned baseline T5Gemma2-2B, TyFlow-2B still improves all four benchmarks, which is the controlled comparison the paper's claims rest on.

Changes made.

- **Added** Table 3 (p. 27): pass@1 of MiMo-7B, Qwen3-14B, and Qwen3-30B-A3B on all four benchmarks, zero-shot and few-shot, with the prompt protocol in the table note.
- **Added** Sec. 6.2.1 (p. 26): the SuFu results and the Java pass@1 comparison with TyFlow-2B, using task prompts without examples for the larger models, together with the data-leakage caveat.

Revised manuscript, Sec. 6.2.1, setup (p. 26) – the setup of the decoder-only comparison

To contextualize absolute performance, Tab. 3 reports three larger decoder-only models (MiMo-7B, Qwen3-14B, Qwen3-30B-A3B), prompted with the same task prompt in all conditions, alongside our two fine-tuned 2B encoder-decoder models. Each prompted model produces one greedy candidate per task: the zero-shot condition uses only the task prompt, and the few-shot condition prepends three solved examples for the Java benchmarks and six examples for SuFu.

Revised manuscript, Sec. 6.2.1, results (p. 26) – the decoder-only results on Java and SuFu

On SuFu the comparison favors TyFlow: none of the three solves a single task zero-shot, and few-shot prompting reaches at most 8 of 58, whereas TyFlow-2B solves 21 [. . .] On Java, given the task prompt without examples, the three larger models solve 34–42 of 67 MBJP tasks, 3–14 of 16 HumanEval-Java tasks, and 38–52 of 103 TransCoder-GFG tasks at pass@1. TyFlow-2B solves 17/67, 8/16, and 31/103, respectively. These results provide context for performance on Java under the respective generation settings. [. . .] we therefore flag a possible data-leakage risk for the Java comparison.

Comment 1.6.

What do the remaining failing programs look like? Are they semantically wrong but well-typed? Do they fail on edge cases? A qualitative analysis of failure modes would significantly strengthen the paper.

Response.

Thanks for this suggestion. Most tasks not solved by TyFlow-2B at pass@1 have compilable candidates whose behavior does not match the task specification; some contain a correct candidate that is not ranked first. We analyze these outcomes in App. C (pp. 42–43), comparing TyFlow-2B with its baseline on all three Java benchmarks using four mutually exclusive categories.

The four categories are:

1. **All-invalid:** no candidate compiles.
2. **Well-typed but wrong:** at least one candidate compiles, but none passes all tests.
3. **Ranking failure:** a correct candidate exists among the ten, but it is not ranked first.
4. **Solved@1:** the top-1 candidate passes all test cases.

Failure analysis of each category:

- **Tasks end without any compilable candidate.** On MBJP, the baseline produces no compilable candidate for four tasks, while TyFlow produces at least one for every task.
- **Compilable programs with incorrect behavior.** In these cases, at least one candidate compiles, but none implements the required behavior to pass all tests. For example, of the 50 MBJP tasks that TyFlow does not solve with its first candidate, 38 fall into this category. This is also the largest remaining failure category on HumanEval-Java and TransCoder-GFG. App. C.2 illustrates how a candidate can satisfy the implemented typing rules but fail to match the task specification. Such errors need not be limited to edge cases.
- **Correct candidates generated but not ranked first.** For these tasks, TyFlow finds a solution within its ten candidates, but the first candidate is incorrect. This occurs on 12 MBJP tasks, one HumanEval-Java task, and 17 TransCoder-GFG tasks. These tasks therefore count as successes at pass@10 but not at pass@1.
- **Tasks solved by the first candidate.** TyFlow improves functional correctness as well as compilability. On MBJP, the first candidate passes all tests for 17 tasks, compared with nine under the baseline. The same improvement appears on HumanEval-Java, from five solved tasks to eight, and on TransCoder-GFG, from 20 to 31.

Changes made.

- **Added** App. C (p. 42): the four-category taxonomy and Table 8 comparing the 2B models on all three Java benchmarks, with an explanation of compilable-but-incorrect candidates and ranking failures.
- **Added** App. C.1–C.2 (pp. 42–43): representative compilation and semantic failures, with the distinction between general logic errors and edge-case failures clarified in C.2.

- **Added** Sec. 6.2.1 (p. 25): a pointer to the failure analysis when discussing candidate ranking and compilable but incorrect programs.

Revised manuscript, App. C (p. 42) – the failure-category results

On MBJP, the baseline produces no compilable candidate for four tasks, while TyFlow produces at least one for every task. On HumanEval-Java, neither model has a task with no compilable candidate; on TransCoder-GFG, both models have two such tasks. Most tasks not solved by TyFlow at pass@1 have at least one compilable candidate but no candidate passing all tests. On MBJP, for example, this accounts for 38 of the 50 tasks not solved at pass@1; it is also the largest remaining failure category on the other two benchmarks. In other cases, a correct candidate is generated but not ranked first: this occurs on 12 MBJP tasks, one HumanEval-Java task, and 17 TransCoder-GFG tasks. These tasks count as successes at pass@10 but not at pass@1.

Revised manuscript, App. C.2, representative case (p. 43) – the representative semantic failure case

The candidate satisfies the implemented typing rules, but its behavior does not match the task specification. This case illustrates a general logic error rather than a failure confined to edge cases; the taxonomy does not separately count edge-case failures.

Revised manuscript, Sec. 6.2.1 (p. 25) – the pointer to failure analysis

The failure analysis in Appendix C identifies ranking errors and compilable but incorrect programs among the remaining failures.

Response to Reviewer 2

We thank the reviewer for the positive assessment of the theoretical foundation, the framework design, and the ablation study.

Comment 2.1.

W1: Limited evaluation benchmarks. The evaluation uses only a limited number of SuFu test tasks and Java test tasks. These are very small test sets, and no confidence intervals or statistical significance tests are reported. In addition, the SuFu language is highly niche. While this is positioned as a “low-resource” evaluation, it’s unclear how findings generalize beyond this specific setting. There is no evaluation on standard, widely-used benchmarks. While the approach requires language-specific tooling (type checker, parser), the absence of comparison on common benchmarks could limit the paper’s impact and make it harder to contextualize the results.

(Addresses W1) Report confidence intervals or evaluate over multiple random train/test splits to establish statistical stability given the small test sets. Consider adding at least one more widely-used benchmark to help contextualize the results.

Response.

Thanks for this comment. [Comment 1.3](#) presents the expanded evaluation and statistical analysis in detail, including the benchmark settings, results, confidence intervals, and paired tests. We summarize the answers to your specific concerns below.

(1) Evaluation on additional benchmarks. We add HumanEval-Java and TransCoder-GFG alongside MBJP. On both added benchmarks, TyFlow-2B improves over its baseline on all four metrics: pass@1, pass@10, FSP, and CER. In particular, pass@10 rises from 37.50% to 56.25% on HumanEval-Java and from 33.01% to 46.60% on TransCoder-GFG. These results show that the gains are not confined to SuFu or the original MBJP test set.

(2) Statistical reliability. At 220M, the small Java test set limits our ability to detect statistically significant improvements: the pass@1 and pass@10 differences are not significant, and only the CER difference is significant. In the 2B experiments, we expand the Java evaluation with two additional benchmarks. On the combined Java test set, improvements on all four metrics are statistically significant. [Comment 1.3](#) gives the confidence intervals and paired-test results.

Changes made.

- **Added** Sec. 6.1.1 (p. 24) and App. B (pp. 39–40): HumanEval-Java and TransCoder-GFG and their construction.
- **Added** Sec. 6.2.1 (p. 25) and Table 2 (p. 26): the results on the two added Java benchmarks, including the pass@10 improvements reported above.
- **Added** App. D: confidence intervals and paired significance tests, detailed and quoted in

Comment 1.3.

Revised manuscript, Sec. 6.1.1 (p. 24) – the two added benchmarks and their splits

HumanEval-Java and TransCoder-GFG are two additional Java benchmarks that broaden the evaluation beyond MBJP. HumanEval-Java adapts the HumanEval problems to Java, and TransCoder-GFG collects the Java programs released with TransCoder; both follow the task format of MBJP. HumanEval-Java uses a fixed split of 146 training and 16 test tasks, and TransCoder-GFG uses a fixed split of 414 training and 103 test tasks.

Revised manuscript, Sec. 6.2.1 (p. 25) – improvements on the added Java benchmarks

TyFlow improves code generation accuracy at both scales and on every benchmark: [...] pass@1 rises [...] 31.25% to 50.00% (HumanEval-Java), and 19.42% to 30.10% (TransCoder-GFG) at 2B; pass@10 improves by 7.46–20.69 points on the same six benchmarks.

Comment 2.2.

W2: Scalability concerns are insufficiently addressed.

The paper does not discuss how the synthesis derivation tree scales with program complexity. Real-world programs could have deep type derivation trees with polymorphism, subtyping, type inference, and overloading resolution, but the Java subset avoids most of these. It is also unclear what runtime overhead the synthesis system imposes during inference, as the dynamic re-encoding at each step and the unification/constraint-checking machinery introduce costs that are never quantified.

(Addresses W2) Report runtime inference time comparisons between TyFlow and the baselines. Discuss, even qualitatively, how the synthesis system would scale to richer type features such as polymorphism with variance, subtyping hierarchies, or ownership types. Report how frequently beam search exhausts all candidates due to pruning and what the fallback behavior is.

Response.

Thanks for this comment. We explain the size of the encoded derivation, the measured runtime cost of each component, and the behavior of beam search below.

(1) Synthesis-tree encoding and richer type systems. We encode a synthesis derivation tree as a sequence of decisions in the order in which its goals are resolved. Each rule selection is represented by one token. Variable assignments are encoded as sequences of constructor and constant tokens. The model therefore generates tokens representing the decisions needed to reconstruct the tree, rather than the full text of every typing judgment.

The resulting sequences are comparable in length to ordinary tokenized code. TyFlow generates an average of 410 tokens per program on SuFu, compared with 405 tokens per program for the ordinary text-based encoding. On Java, the corresponding averages are 129 and 142 tokens. A rule token incorporates syntax that would otherwise require separate code tokens, such as punctuation. The same encoding applies when extending the type system as discussed in [Comment 1.1](#): each added rule still uses one token per application. Adding more available rules

does not mean emitting all of them, so a larger rule set does not itself cause an explosion in sequence length of our synthesis trees. Sequence length depends on how many rule applications and variable assignments the derivation requires.

(2) Runtime overhead of each component. Our ablation study uses the 220M model on SuFu in four cumulative configurations: the base model without checks, then adding syntactic pruning, type pruning, and the dynamic typing context in turn. A decoding step extends the active hypotheses by one token, including the enabled checks and context update. In this order, average time per step is $31.1 \rightarrow 32.4 \rightarrow 43.7 \rightarrow 59.6$ ms. Syntactic pruning is inexpensive: checking whether a proposed token fits the expected syntax adds only 1.3 ms per step. Type pruning adds 11.3 ms because it invokes the type checker on candidate extensions; encoding the updated typing context adds a further 15.9 ms because the evolving synthesis goal must be processed by the encoder.

Average time per task is $5.70 \rightarrow 5.34 \rightarrow 11.07 \rightarrow 15.62$ s in the same configuration order. The overall trend is similar: type checking and context encoding account for most of the additional cost. Syntactic pruning is the exception. By eliminating invalid decisions early, it keeps search on well-formed derivations that finish in fewer steps. This reduction outweighs its small per-step overhead, lowering average task time from 5.70 to 5.34 s.

(3) Beam-search behavior. As explained in [Comment 1.2](#), a finite beam can still discard all correct continuations. If the active beam is exhausted, search stops and retains any completed candidates. If none has been completed, the run produces no solution. In the instrumented run, however, the active beam contained 7.97 hypotheses on average with a maximum width of ten, and never became empty.

Changes made.

- **Added** Sec. 6.2.4 and Table 6 (p. 29): the token-length comparison between decision sequences and ordinary code; Sec. 6.3 (p. 30) discusses richer type-system extensions, also explained in [Comment 1.1](#).
- **Added** Sec. 6.2.2 and Table 4 (pp. 27–28): the runtime measurements for the four component configurations, including average per-step and per-task times.
- **Added** Sec. 6.2.2 (p. 27): the discussion of finite-beam search and the observed active-beam size (7.97 on average, with no empty active beam in the instrumented run).

Revised manuscript, Sec. 6.2.4 (p. 29) – decision-sequence length compared with code tokens

It is even comparable to, or slightly shorter than, the plain code tokens alone (405 on SuFu, 142 on Java): a type derivation rule absorbs the redundant syntactic elements of the code (such as equals signs and semicolons) into a single rule token, so our representation adds little and sometimes removes tokens.

Revised manuscript, Sec. 6.2.2, “Runtime cost” (p. 28) – the runtime measurements

Each component increases the per-step cost ($31.1 \rightarrow 32.4 \rightarrow 43.7 \rightarrow 59.6$ ms). Syntactic pruning adds little (roughly +4%); type pruning and the dynamic typing context cost considerably more, as the former invokes the type checker at every step and the latter additionally re-encodes the evolving synthesis goal. The per-task totals move by -6.38% / $+94.00\%$ / $+173.88\%$ ($2.74\times$ base). [. . .]

Revised manuscript, Sec. 6.2.2 (p. 27) – the beam-exhaustion observation

A finite beam can discard all correct prefixes, and the retained hypotheses may later have no valid continuation, leaving the active beam empty. [. . .] We measure the effective branching factor as the number of active hypotheses present at each decoding step after pruning. With beam 10, this number averages 7.97. [. . .] In our experiments, however, the active beam never became empty.

Revised manuscript, Sec. 6.3, “Type-system expressiveness” (p. 30) – the scope of richer type-system extensions

For features that can be represented with first-order terms, such as many forms of subtyping, overloading, effects, and type classes, we need to implement the typing rules and solvers and then measure their cost. The current solver cannot handle a typing rule that requires higher-order unification, as discussed in Sec. 3. Our experiments do not test either type of extension. We leave them for future work.

Comment 2.3.

W3: Comparison with modern LLMs is absent. The baselines are CodeT5-220M and T5Gemma2-2B, and both are relatively small encoder-decoder models. There is no comparison with decoder-only models which are much more common in today’s community. Additionally, no model larger than 2B parameters is evaluated. While the paper demonstrates clear improvements over its chosen baselines, it remains an open question whether larger models might already mitigate type errors sufficiently through scale alone. This would contextualize the practical value of the proposed approach.

(Addresses W3) Include at least a zero-shot or few-shot evaluation of a modern decoder-only model on the same benchmarks. Additionally, discuss whether the core ideas could be adapted to decoder-only architectures, even if full integration is left to future work.

Response.

Thanks for this comment. We summarize the larger-model comparison reported in [Comment 1.5](#), then explain how TyFlow could be adapted to a decoder-only model.

(1) The comparison with larger decoder-only models. [Comment 1.5](#) gives the settings and results for MiMo-7B, Qwen3-14B, and Qwen3-30B-A3B. These pretrained models have a strong Java foundation, which helps explain their higher performance on MBJP. They solve 34–42 of 67 MBJP tasks, compared with 17 for TyFlow-2B. However, their performance is much lower on the low-resource language SuFu, where none solves a task without examples, and few-shot prompting reaches at most 8 of 58, compared with 21 for TyFlow-2B.

Model size needs to be considered together with the available language-specific training data. The larger prompted models do not outperform TyFlow-2B on SuFu, showing that more parameters alone do not ensure better results in this setting.

(2) Adapting TyFlow to a decoder-only model. The decision-sequence representation can be used unchanged: a decoder-only model can predict the next rule or variable-assignment token from the prompt and previous decisions. The synthesis engine would still apply that decision, update the derivation tree, and check the constraints. The part that needs adaptation is the dynamic typing context. In the current encoder-decoder design, the encoder processes the changing synthesis goal separately from the decision sequence.

One adaptation would supply the current typing context as temporary input after the decision prefix. The decoder would process this context to predict the next decision, after which the synthesis engine would update the goal. Before the following prediction, the old context would be removed and replaced with the updated one. Its key-value cache entries would therefore need to be recomputed at every step rather than reused. Thus, the representation and synthesis rules carry over, while context input and cache management require changes to the decoding procedure. Sec. 6.3 describes this adaptation direction; implementation and runtime evaluation remain future work.

Changes made.

- **Added** Table 3 (p. 27) and Sec. 6.2.1 (p. 26): the decoder-only comparison, detailed in [Comment 1.5](#).
- **Added** Sec. 6.3, “Dependence on the encoder-decoder architecture” (p. 29): what a decoder-only adaptation requires.

Revised manuscript, Sec. 6.2.1 (p. 26) – the decoder-only comparison

On SuFu the comparison favors TyFlow: none of the three solves a single task zero-shot [. . .] On Java, given the task prompt without examples, the three larger models solve 34–42 of 67 MBJP tasks, 3–14 of 16 HumanEval-Java tasks, and 38–52 of 103 TransCoder-GFG tasks at pass@1. TyFlow-2B solves 17/67, 8/16, and 31/103, respectively.

Revised manuscript, Sec. 6.3, “Dependence on the encoder-decoder architecture” (p. 29) – the decoder-only adaptation path

The decision-sequence representation and synthesis rules can be used unchanged with a decoder-only model. One adaptation would append the current typing context to the decision prefix as temporary input for predicting the next decision, then replace it with the updated context before the following prediction. This requires recomputing the context’s key-value entries at each step and adapting cache management to the changing context. Implementing this decoding procedure and evaluating its runtime are directions for future work.

Comment 2.4.

W4: The CHC generality claim is overclaimed relative to the evidence. The paper repeatedly emphasizes that CHCs can encode “any Turing-computable function” and that the framework generalizes to “arbitrary logical constraints.” However, the evaluation only covers type correctness for two languages with relatively simple type systems. No evidence is provided for richer constraints.

The gap between the theoretical generality and empirical validation is large.

(Addresses W4) Either provide a case study demonstrating a constraint richer than basic type correctness (e.g., ownership, effects, refinement types), or explicitly scope the generality claims to what is empirically demonstrated and acknowledge the gap as a limitation.

Response.

Thanks for this comment. TyFlow’s generality comes from its construction at the CHC level: the transformation from proof rules to synthesis rules is not specific to Java or SuFu. Extending it to another constraint family requires expressing the relevant rules in the supported form and providing the required constraint-checking procedures.

Our experiments establish the benefits of this construction for type correctness in Java and SuFu. Extensions such as ownership or effect constraints would additionally require implementing their rules and solvers, preparing training examples, and evaluating both correctness and runtime. We therefore focus on type correctness in Java and SuFu in this paper and plan to explore other constraints in future work. The revised manuscript clarifies the evaluated scope and the extension path, including in Sec. 6.3 (Limitations and Future Work).

Changes made.

- **Clarified** Sec. 1 (p. 4): the CHC-level construction, its solver requirements, and its evaluation for type correctness in Java and SuFu.
- **Clarified** Sec. 7.2 (p. 31): the extension from grammar-based representations to type-correctness proofs.
- **Clarified** Sec. 8 (p. 31): richer constraint families as a direction for future implementation and evaluation.
- **Added** Sec. 6.3, “Type-system expressiveness” (p. 30): the implementation and cost evaluation required to support richer type features, including effects.

Revised manuscript, Sec. 1 (p. 4) – the scoped CHC claim in the introduction

Notably, all proof rules in our system are formulated as constrained Horn clauses (see Sec. 3.1.1). This representation subsumes prior work on syntactic correctness and provides a common basis for constructing synthesis rules from proof rules. The construction applies to rules in the supported CHC form with suitable unification and constraint-checking procedures. We instantiate and evaluate the framework for type correctness in Java and SuFu.

Revised manuscript, Sec. 7.2 (p. 31) – the extension from grammar-based representations to type-correctness proofs

Grammar-based encoding can be viewed as a special case of our framework when grammar rules are expressed as CHCs. TyFlow extends this rule-based representation from syntactic structure to type-correctness proofs, with implementations and evaluations in Java and SuFu.

Revised manuscript, Sec. 8 (p. 31) – the scoped CHC claim in the conclusion

Beyond type correctness, TyFlow’s CHC foundation suggests a route toward richer constraint families, such as safety verification and the generation of other structural data; instantiating and evaluating these extensions remains future work.

Revised manuscript, Sec. 6.3, “Type-system expressiveness” (p. 30) – requirements for richer type features

For features that can be represented with first-order terms, such as many forms of subtyping, overloading, effects, and type classes, we need to implement the typing rules and solvers and then measure their cost.

Response to Reviewer 3

We thank the reviewer for the positive assessment of the typing-based extension of grammar-guided generation and of the rigor of the isomorphism between type derivations and synthesis derivations.

Comment 3.1.

Missing comparisons against token-level constrained decoding works such as SynCode and Copiloting the Copilots. Those works also guide LLM generation, but they are better than rejection sampling because they guide the LLM at the token level. To demonstrate the robustness of TyFlow, comparisons against those works are important.

Response.

Thanks for this suggestion. We added comparisons using SynCode and Repilot (*Copiloting the Copilots*) on MBJP, alongside rejection sampling and iterative compiler repair. We reuse their released constraint engines and adapt the surrounding generation code to our model and task format. The experiments compare these decoding-time mechanisms with TyFlow’s type-guided generation; the implementations and evaluated configurations are described below.

SynCode. We reuse SynCode’s released grammar-masking code and Java grammar with T5Gemma2 on MBJP. SynCode masks tokens that would violate the grammar at each decoding step to ensure syntactically well-formed outputs. In the evaluated compile-safe configuration, CER falls from 26.12% to 21.34%, while the solved sets remain unchanged.

Repilot. We reuse Repilot’s modified Eclipse JDT completion engine and adapt it to T5Gemma2 on MBJP. During generation, the partial Java program is maintained as a JDT document, and completion queries guide token pruning. Following the original policy, punctuation and keywords bypass these queries. CER changes from 26.12% to 25.97%, and the solved sets remain unchanged.

Rejection sampling is the only external method that increases the number of solved tasks, adding three solved tasks at pass@10 and one at pass@1, but neither difference is statistically significant.

TyFlow-2B solves 17/67 at pass@1 and 29/67 at pass@10 with only 3/670 compilation errors, the best result among all compared methods. TyFlow enforces the modeled type constraints during generation and reduces CER to 0.45%, rather than filtering or repairing what the model already produces. These results suggest that when the model assigns too little probability to correct programs, decoding-time constraints alone offer limited gains, making a suitable representation and training is more effective way to improve the learned distribution.

Changes made.

- **Rewrote** Sec. 6.2.3 (p. 28): the comparison of external generation mechanisms with TyFlow and the results in Table 5.
- **Clarified** Sec. 6.2.3 (p. 28): reuse of the released SynCode and Repilot/JDT components and how their grammar masking and completion-guided token pruning are applied to MBJP generation.

Revised manuscript, Sec. 6.2.3 (p. 28) – the five-method setup and shared protocol

On MBJP, four such methods are applied on top of the same frozen T5Gemma2-2B checkpoint selected in RQ1, adding their own pruning or repair mechanisms on top: *rejection sampling*, *SynCode*, *Repilot*(*Copiloting the Copilots*), and *iterative compiler repair*. All methods are evaluated on the same 67 tasks with 10 candidates each. Because rejection sampling and repair need diverse draws rather than beam output, the ordinary control uses the temperature-sampling decoding protocol (temperature 0.8, top- p 0.95), so its scores (14.93/23.88) differ from the beam-10 row of Tab. 2 (13.43/32.84); the TyFlow-2B row is identical in both tables.

Revised manuscript, Sec. 6.2.3 (p. 28) – reuse and adaptation of SynCode

For SynCode, we reuse its released grammar-masking code and Java grammar with T5Gemma2 on MBJP. At each step it masks the tokens that would violate a given grammar, so every output is syntactically well-formed.

Revised manuscript, Sec. 6.2.3 (p. 28) – reuse and adaptation of Repilot/JDT

For Repilot, we reuse its modified Eclipse JDT completion engine and adapt it to T5Gemma2 on MBJP. During generation, the partial Java program is maintained as a JDT document, and completion queries guide token pruning. Following the original policy, punctuation and keywords bypass these queries.

Revised manuscript, Sec. 6.2.3 (pp. 28–29) – the outcome and the Type Explicitness conclusion

None of the four external baselines significantly improves functional correctness over the ordinary control: rejection sampling adds three solved tasks at pass@10 (19 vs. 16 of 67) and one at pass@1 (11 vs. 10; McNemar $p = 1.00$ and $p = 0.25$); SynCode, Repilot, and iterative repair leave the solved sets exactly unchanged (10/67 and 16/67) [...] TyFlow-2B solves 17/67 at pass@1 and 29/67 at pass@10 with 3 of 670 candidates failing to compile, the best of all compared methods. The evaluated external mechanisms constrain or revise generated programs without updating the underlying model parameters, whereas TyFlow explicitly encodes the typing rules into the model and helps it better learn the program distribution, supporting the value of the Type Explicitness property in Sec. 1.

Comment 3.2.

Missing comparison against iterative refinement approaches: Compared to rejection sampling, iterative refinement approaches regenerate the program with feedback (in this case, the typing failure feedback), which is strictly stronger than rejection sampling as it incorporates typing failure feedback. To demonstrate TyFlow’s robustness, the authors should compare it with this line of work.

Response.

Thanks for this comment. Iterative compiler repair is now one of the five compared methods: javac diagnostics are fed back to the model, which regenerates the candidate for at most two rounds; the repair agent is the un-fine-tuned T5Gemma2-2B, since the fine-tuned model never learned to consume diagnostics. We cap repair at two rounds per candidate to bound the additional generation cost and stop earlier if the candidate compiles.

Compiler feedback improves compilability in our experiments: the number of candidates that fail to compile falls from 175 to 116 out of 670. However, none of the 59 candidates repaired to compile passes the functional tests, so the solved sets remain unchanged at 10/67 for pass@1 and 16/67 for pass@10. Compiler diagnostics identify compilation errors, but do not establish whether a program implements the intended behavior. The feedback therefore helps repair compilation failures without necessarily correcting the program's functionality.

Rejection sampling explores additional candidates and can find functionally correct programs through repeated generation. In our experiments, it adds one solved task at pass@1 and three at pass@10, although neither gain is statistically significant. These results show that the additional compiler feedback is useful for compilability, but does not automatically lead to higher functional correctness than repeated sampling.

Changes made.

- **Added** Sec. 6.2.3 (pp. 28–29): the repair protocol (javac feedback, at most two rounds, un-fine-tuned agent), its measured outcome, and an explanation of why compiler feedback improves compilability without adding solved tasks.

Revised manuscript, Sec. 6.2.3 (p. 28) – the iterative-repair protocol

Iterative compiler repair feeds javac diagnostics back to the model and regenerates the candidate for at most two rounds; the repair agent is the un-fine-tuned T5Gemma2-2B, since the fine-tuned model never learned to consume diagnostics. We cap repair at two rounds per candidate to bound the additional generation cost and stop earlier if the candidate compiles.

Revised manuscript, Sec. 6.2.3 (pp. 28–29) – improved compilability without additional solved tasks

[...] iterative repair reduces compilation failures from 175 to 116 of 670 candidates, but none of the 59 candidates repaired to compile passes the functional tests. Compiler diagnostics help resolve compilation errors, but do not establish whether a program implements the intended behavior, explaining why improved compilability does not translate into additional solved tasks here.

Comment 3.3.

The TyFlow system requires multiple LLM calls to obtain all synthesis decisions. To show the practicality of TyFlow, it is important to provide an analysis of the system's cost for generating the program.

Response.

Thanks for this question. We first clarify the base case, then explain what our method adds on top of it.

(1) The base case is a single encoder–decoder call. The system is not a multi-LLM pipeline: the encoder processes the prompt once, and the decoder generates the output token by token, reusing the encoder’s representation throughout. There are no repeated LLM calls to separate models.

(2) What our method adds on top of this base. During decoding, we add syntactic pruning (checking each generated token against the grammar), type pruning (checking completed AST fields against the type system), and a dynamic typing context (re-encoding the synthesis goal as it evolves). Each of these adds moderate per-step cost, but the encoder’s computation is fully reused, so the total does not grow proportionally.

(3) The measured cost. RQ2 now times the same 220M model in the four component configurations, reporting per-step ($31.1 \rightarrow 32.4 \rightarrow 43.7 \rightarrow 59.6$ ms) and per-task totals ($5.70 / 5.34 / 11.07 / 15.62$ s, i.e. $2.74\times$ the unconstrained base). Each increment has a stated cause (per-step type checking; re-encoding the evolving synthesis goal). For the compared external methods, the RQ3 setup states each method’s computational budget.

Changes made.

- **Added** Sec. 6.2.2 (p. 27): per-step and per-task runtime for the four configurations (Table 4).
- **Added** Sec. 6.2.3 (p. 28): the computational budget of each compared external method.

Revised manuscript, Sec. 6.2.3 (p. 28) – the rejection-sampling budget

Rejection sampling repeatedly draws candidates and keeps those that pass Java compilation, for up to ten ten-draw rounds (100 candidates) per task.

Revised manuscript, Sec. 6.2.2, “Runtime cost” (p. 27) – the runtime measurements

Syntactic pruning adds little (roughly +4%); type pruning and the dynamic typing context cost considerably more, as the former invokes the type checker at every step and the latter additionally re-encodes the evolving synthesis goal.

Comment 3.4.

Limited Java Performance: Although TyFlow brings significant improvement on SuFu, its performance boost on Java is marginal (from 10.45% to 11.94%).

Response.

Thanks for this observation. [Comment 1.4](#) explains the Java results and gives context from prior work on code representations. The pretrained baseline already has a strong Java foundation and produces fewer compilation errors than on SuFu, leaving fewer errors for type constraints to

address. At 220M, Java pass@10 improves from 20.90% to 28.36%, and CER falls from 35.52% to 1.53%. At 2B, all four metrics improve on each of the three Java benchmarks (Comment 1.3), showing consistent benefits over the corresponding baseline.

Changes made.

- **Added** Sec. 6.2.1 (p. 25): the Java results at both model scales and the explanation of the 220M gains across metrics.

Revised manuscript, Sec. 6.2.1 (p. 25) – the Java results at both scales

TyFlow improves code generation accuracy at both scales and on every benchmark: pass@1 rises from 24.14% to 37.93% (SuFu) and from 10.45% to 11.94% (MBJP) at 220M, and from 25.86% to 36.21% (SuFu), 13.43% to 25.37% (MBJP), 31.25% to 50.00% (HumanEval-Java), and 19.42% to 30.10% (TransCoder-GFG) at 2B; pass@10 improves by 7.46–20.69 points on the same six benchmarks.

Revised manuscript, Sec. 6.2.1 (p. 25) – the Java gains across correctness and compilation metrics

At 220M, the Java baseline has a lower CER than the SuFu baseline (35.52% versus 83.10%), leaving fewer compilation errors for type constraints to address. On MJB, TyFlow raises pass@10 from 20.90% to 28.36% and reduces CER to 1.53%; the smaller pass@1 gain indicates that finding a correct candidate does not always make it the top-ranked output. The failure analysis in Appendix C identifies ranking errors and compilable but incorrect programs among the remaining failures.

Comment 3.5.

Limited model choices: The TyFlow system was mainly evaluated on small models. Evaluation on larger models (decoder-only, billions of parameters) is unexplored. The authors should consider extending the evaluation to larger models.

Response.

Thanks for this comment. We added zero- and few-shot evaluations of three larger decoder-only models. The results and adaptation requirements are also discussed in Comment 1.5 and Comment 2.3.

The added comparison. Three open-weight decoder-only models (MiMo-7B, Qwen3-14B, Qwen3-30B-A3B; 7B–30B parameters) are evaluated on all four benchmarks, zero- and few-shot, under a separate prompt-based protocol kept distinct from the trained results (Table 3, p. 26); the evaluation now spans five model scales.

The results. On Java, the prompted decoder-only models solve 33–45 of 67 MJB tasks, compared with 17 of 67 for TyFlow-2B. On SuFu, the low-resource setting the method targets, none of the three solves a single task zero-shot, and few-shot prompting reaches at most 8 of 58, against 21 of 58 for TyFlow-2B.

The conclusions. The comparison shows that TyFlow-2B can outperform these larger prompted models on a low-resource language. The representation also carries over to decoder-only

models; adapting the dynamic typing context requires recomputing its key-value entries at each step, as explained in [Comment 2.3](#). Implementing and evaluating this adaptation remains future work.

Changes made.

- **Added** Table 3 (p. 26): the three decoder-only models on all four benchmarks.
- **Added** Sec. 6.3 (p. 29): the requirements for a decoder-only adaptation.

Revised manuscript, Sec. 6.2.1, Table 3 and its note (p. 26) – the decoder-only comparison and its protocol

To contextualize absolute performance, Tab. 3 reports three larger decoder-only models (MiMo-7B, Qwen3-14B, Qwen3-30B-A3B), prompted with the same task prompt in all conditions, alongside our two fine-tuned 2B encoder-decoder models. Each prompted model produces one greedy candidate per task [. . .] Cell entries are solved tasks at pass@1 for all models; the 0-shot condition uses the same prompt as the two fine-tuned models.

Comment 3.6.

Generalizability concern: TyFlow relies strongly on the typing system of the language, but how would it work on a dynamically typed language such as Python? Thus, the author should acknowledge this in the threat to validity to make this paper more robust.

Response.

Thanks for this comment. TyFlow extracts training decision sequences automatically from type derivations in the two evaluated languages. Applying this pipeline to Python requires a suitable type system and a way to recover its derivations automatically, since types may depend on control flow and runtime information. The current implementation targets statically typed languages, and adaptation to dynamically typed languages remains future work. We now explain this requirement in Sec. 6.3.

Changes made.

- **Added** Sec. 6.3, “Dynamically typed languages” (p. 30): the requirements for adapting automatic training-data extraction to dynamically typed languages.

Revised manuscript, Sec. 1, “Data Usability” (p. 3) – the Data Usability property

The decision sequences of S and source code should be automatically and bidirectionally convertible, enabling both the extraction of decision sequences from existing programs and the reconstruction of programs from generated decision sequences.

Revised manuscript, Sec. 6.3, “Dynamically typed languages” (p. 30) – the dynamic-language limitation

The current design of TyFlow relies on a type system in which the type derivation of a program can be computed automatically from the program text, so that existing code can be converted into our decision-sequence format. Dynamically typed languages such as Python still have typing rules to a degree, but the type derivation cannot be read off the program text alone, since types depend on control flow and runtime information. [. . .] Parsing and adapting such languages is left to future work.

Other Changes beyond the Reviewers' Requests

- **Re-trained results.** The results in the submitted version were produced in 2025 in a different environment, and those checkpoints and evaluation records are not available on the server used for this revision; every model reported here was therefore re-trained with the same framework and the same code, and all evaluations were re-run. The new run supports the conclusions of the submitted version; the differences are small (most values move by less than two percentage points; the largest is the 2B SuFu row, where the re-measured baseline is 25.86% (submitted: 29.31%) and the reported TyFlow-2B value is 36.21% (submitted: 43.10%); TyFlow still exceeds the baseline on all four metrics); the intervals and paired tests of Appendix D are computed on the new runs, so the significance statements apply to the numbers printed in the tables.
- **Errata.** Algorithm 1 now composes substitutions as $\sigma_{i-1} \dots \sigma_0 \sigma$; the statement after Lemma 2 reads $P(\theta(\bar{i}))$; and the proof of Lemma 4 defines $\theta_c = \sigma_n \dots \sigma_0 \sigma$. The method and the theorems are unchanged.
- **FSP definition.** Sec. 6.1.3 (p. 24): a task with no correct candidate contributes the candidate-list length, a convention the submitted text left implicit.

We hope that the revision addresses all concerns and look forward to the reviewers' feedback.

Sincerely,
The Authors